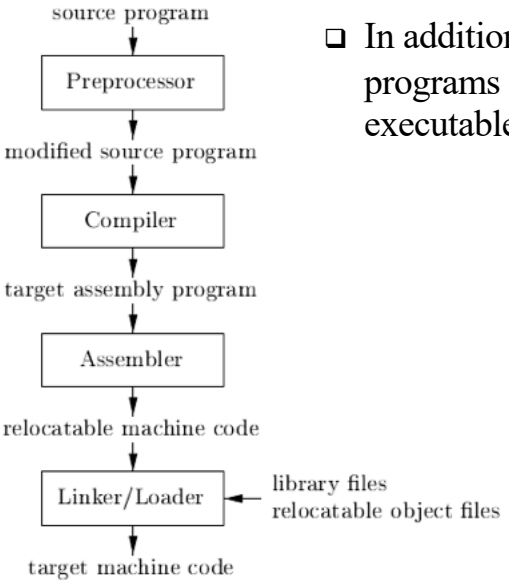
Compiler techniques for source code analysis

The compiler context



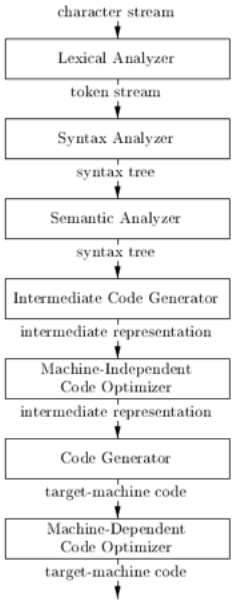
- In addition to a compiler, several other programs may be required to create an executable target program

The structure of a compiler

- The analysis part (front-end) breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program.
- The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part.

Symbol Table

- The synthesis part (back-end) constructs the desired target program from the intermediate representation and the information in the symbol table
- We are only interested in the front-end techniques

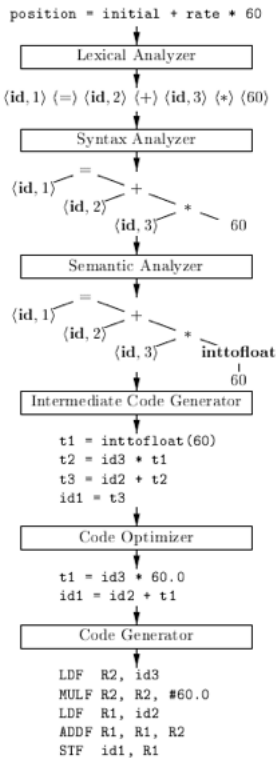


Lexical analysis

- The first phase of a compiler is ***lexical analysis*** or ***scanning***.
- The lexical analyzer reads the strean of characters making up the source program and groups the characters into meaningful sequences called ***lexemes***.

1	position	···
2	initial	···
3	rate	···

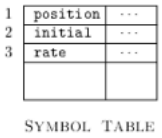
SYMBOL TABLE



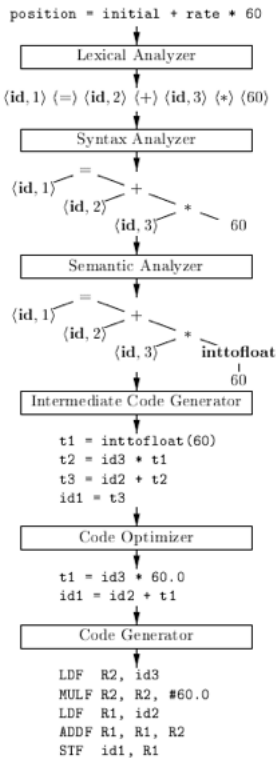
- For each lexeme, the lexical analyzer produces as output a token of the form ❮*token-name*; *attribute-value*❯ that it passes on to the subsequent phase, syntax analysis
- In the token, the first component *token-name* is an abstract symbol that is used during syntax analysis, and the second component attribute-value points to an entry in the symbol table for this token.
- Information from the symbol-table entry is needed for semantic analysis and code generation.

Syntax analysis

- The second phase of a compiler is ***syntax analysis*** or ***parsing***.
- The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.
- A typical representation is a ***syntax tree*** in which each interior node represents an operation and the children of the node represent the arguments of the operation.



SYMBOL TABLE

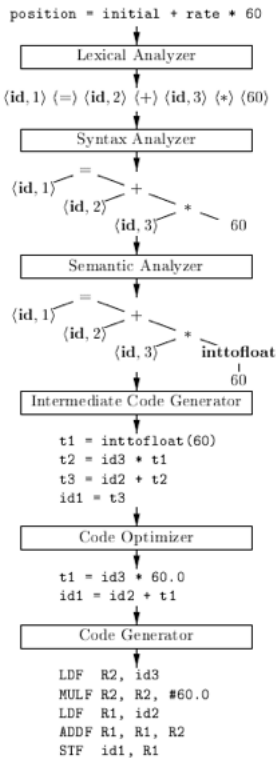


Semantic analysis

- The ***semantic analyzer*** uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.

1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE



- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.
- An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.
- The language specification may permit some type conversions called coercions.

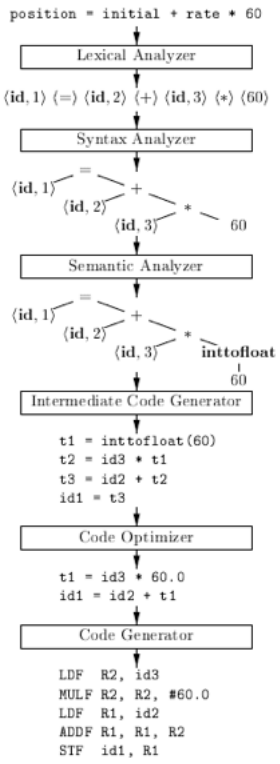
Intermediate Code Generation

- In the process of translating a source program into target code, a compiler may construct one or more intermediate representations, which can have a variety of forms.

1	position	···
2	initial	···
3	rate	···

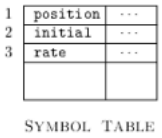
SYMBOL TABLE

- Syntax trees are a form of intermediate representation commonly used during syntax and semantic analysis.
- Many compilers also generate an explicit low-level or machine-like intermediate representation, which we can think of as a program for an abstract machine.
  - ✓ It should be easy to produce and easy to translate into the target machine
  - ✓ Example: *three-address code* consists of a sequence of assembly-like instructions with three operands per instruction, where each operand can act as a register.
- We are interested in producing intermediate representations that can be used for source code analysis

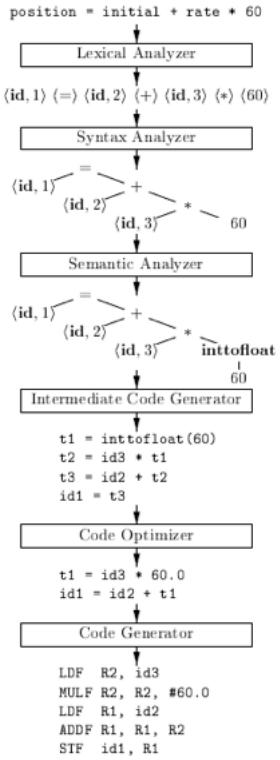


Code Optimization

- The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.
- Usually, better means faster, but it can be also shorter, more energy efficient, and so on.
- Examples are moving constant type conversion at compile time or constant propagation.



SYMBOL TABLE



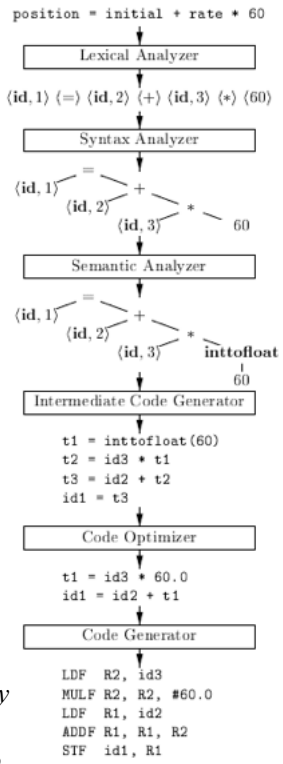
Code Generation

- The code generator takes as input an intermediate representation of the source program and maps it into the target language.

1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE

- If the target language is machine code, registers or memory locations are selected for each of the variables used by the program. Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.
- A crucial aspect of code generation is the judicious assignment of registers to hold variables.
- The first operand of each instruction specifies a destination.
- The F in each instruction tells us that it deals with floating-point numbers.



LD ➔ *load*
MUL ➔ *multiply*
ADD ➔ *add*
ST ➔ *store to*

Symbol Table Management

- An essential function of a compiler is to record the variable names used in the source program and collect information about various attributes of each name.
- These attributes may provide information about the storage allocated for a name, its type, its scope (where in the program its value may be used), and in the case of procedure names, such things as the number and types of its arguments, the method of passing each argument (for example, by value or by reference), and the type returned.
- The symbol table is a data structure containing a record for each variable name, with fields for the attributes of the name.
- The data structure should be designed to allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly.

Source code static analysis

- How do we build static analyzers?
- How do we define the rules for lexical, syntax, and semantic analysis?
- Programs are expressed in programming languages …

Language

- A ***language*** is a set of strings defined on a given alphabet V
  - ✓ by enumeration (… if it is finite)
  - ✓ by set of rules
  - ✓ by a grammar
- The ***cardinality*** of a language is the number of strings it contains
  - ✓ $V = \{0, 1\}$ *alphabet* (*dictionary*)
  - ✓ $L_1 = \{\varepsilon, 0, 00, 000, 0000, 00000\}$ $|L_1| = 6$
  - ✓ $L_2 = \{1, 01, 001, 0001, 00001, \ldots\} = \{0^n 1 \mid n \geq 0\}$ $|L_2| = \infty$
  - ✓ ε is the empty string
- Languages are sets, then we can define operators from set theory
  - ✓ Union: $L_x \cup L_y = \{z \mid z \in L_x \vee z \in L_y\}$
  - ✓ Intersection: $L_x \cap L_y = \{z \mid z \in L_x \wedge z \in L_y\}$
  - ✓ Difference: $L_x - L_y = \{z \mid z \in L_x \wedge z \notin L_y\}$

Concatenation, Closure, and Linguistic Universe

- Concatenation and Closure
  - ✓ Concatenation: $L_x \cdot L_y = L_x L_y = \{xy \mid x \in L_x \wedge y \in L_y\}$
  - ✓ N-th power: $L^n = L \cdot L \cdot \ldots L$ (n times) – $L^1 = L$ – $L^0 = \{\varepsilon\}$
  - ✓ Closure: $L^* = \cup_{i=0}^{\infty} L^i$
  - ✓ Positive closure: $L^+ = \cup_{i=1}^{\infty} L^i$

- Given an alphabet V, the ***Linguistic Universe*** on V, referred as V*, is the set of all strings that can be built with symbols in V, including the empty string

- A ***language*** is a subset of the linguistic universe

$$L(V) \subseteq V^*$$

Grammars

- A ***grammar*** is defined by a *set of rules* that allow to generate all the strings in a given language

- A ***grammar*** is a tuple $G = \{T, N, P, S\}$, where:

  - ✓ T is the alphabet of terminal symbols (appearing in the language strings)
  - ✓ N is a (finite) set of non terminal symbols (the syntactic categories)
  - ✓ $N \cap T = \emptyset$
  - ✓ P is the set of grammar rules (production rules)
  - ✓ $S \in N$ is the start symbol (sentence symbol)

Production Rules and Derivation

- A ***production rule*** is a relation between a pair of strings

$$\alpha \longrightarrow \beta$$

  where $\alpha \in (T \cup N)^+$ contains at least a symbol in N and $\beta \in (T \cup N)^*$

  The production rule states that α (the left side) can be replaced (rewritten) by β (the right side)

- The generative mechanism defined by the grammar is based on the ***derivation operation***: a substring corresponding to the left side of a rule is replaced by the string on the right side

T = {a,b,c}
N = {A,B,C}

A → abc
A → aBbc
Bb → bB
Bc → Cbcc
bC → Cb
aC → aaB
aC → aa

production rules

$L(G) = \{a^n b^n c^n \mid n \geq 1\}$

$A \Rightarrow aBbc \Rightarrow abBc \Rightarrow abCbcc$
$\Rightarrow aCbbcc \Rightarrow aabbcc$

The language of G

- If α can be derived in 0 or more steps from the start symbol S of a grammar G ($S \overset{*}{\Rightarrow} \alpha$), we say that α is a ***sentential form*** of G
  - ✓ Note that a sentential form may contain both terminals and nonterminals, and may be empty.
- A ***sentence*** of G is a sentential form with no nonterminals.
- Given a grammar G, the generated language is the set of sentences that can be derived in 0 or more steps from S

$$L_G = \{x \in T^* \mid S \overset{*}{\Rightarrow} x\}$$

Example: grammar for the arithmetic expressions

T = {num,+,*,(,)}
N = {E}
S = E

E → (E)
E → num
E → E * E
E → E + E

start symbol

E ⇒ E * E ⇒ (E) * E ⇒ (E + E) * E
⇒ (num + E) * E ⇒ (num+num)* E
⇒ (num + num) * num

language phrase

Classes of generative grammars (Chomsky Hierarchy)

- Classification based on the structure of the production rules
  - ✓ Type-0 grammars (unrestricted grammars)
    - » Only restriction: the left side of the production rules must contain at least one non terminal symbol
    - » They generate all languages that can be recognized by a Turing machine
  - ✓ Type-1 grammars (context-sensitive grammars)
    - » Structure of the production rules: $\phi X \gamma \longrightarrow \phi \alpha \gamma$, where $\phi, \gamma \in (T \cup N)^*$, $X \in N$, and $\alpha \in (T \cup N)^+$
    - » The replacement of X by α can only take place in the context of ϕ and γ, i.e. when X appears between the strings ϕ and γ
  - ✓ Type-2 grammars (context-free grammars)
    - » Structure of the production rules: $X \longrightarrow \alpha$, where $X \in N$, and $\alpha \in (T \cup N)^*$
  - ✓ Type-3 grammars ([right] regular grammars)
    - » Structure of the production rules: $X \longrightarrow sY$ or $X \longrightarrow s$, where $X,Y \in N$, and $s \in T^*$

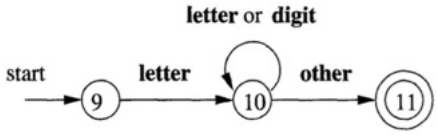
Regular languages

- Regular languages can be described by different equivalent formal models
  - ✓ Finite State Automata (FSA)
  - ✓ Regular Grammars (RG)
  - ✓ Regular Expressions (RE)
- Each formalism is suited for a specific task
  - ✓ FSA defines a ***recognizer*** that can be used to determine if a string belongs to a given regular language
  - ✓ RG defines a generative model for the strings in the language
  - ✓ RE describes the structure of the strings in the language (***pattern***)

Example: Identifiers

- RE
  - ✓ A | B | … | Z | a | b | …| z | (A | B | … | Z | a | b | …| z | 0 | 1 | … | 9)*
  - ✓ or [A-Za-z] ([A-Za-z] | [0-9])*

- RG + RE
  - ✓ letter ⟶ A | B | … | Z | a | b | …| z | or letter ⟶ [A-Za-z]
  - ✓ digit ⟶ 0 | 1 | … | 9 or digit ⟶ [0-9]
  - ✓ id ⟶ letter (letter | digit)*
- FSA



Lexical analysis

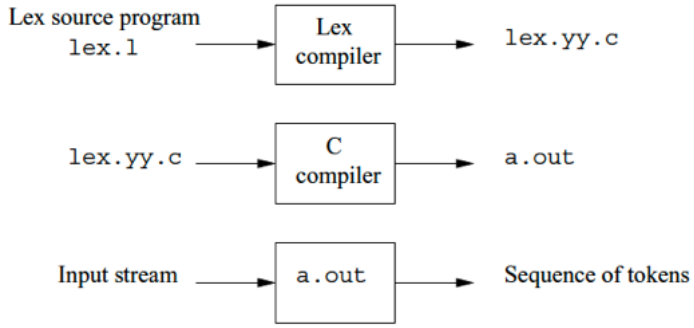
- The main task of the ***lexical analyzer*** is to read the input characters of the source program, group them into lexemes, and produce as output a sequence of tokens for the lexemes in the source program

  - ✓ ***Token***: a pair consisting of a token name and an optional attribute value. The token name is an abstract symbol representing a kind of lexical unit, e.g., a particular keyword, or a sequence of input characters denoting an identifier.
  - ✓ ***Pattern***: a description of the form that the lexemes of a token may take (e.g. through regular expressions)
  - ✓ ***Lexeme***: a sequence of characters in the source program that matches the pattern for a token and identified by the lexical analyzer as an instance of that token.

Building lexical analyzers

- Start with a description of the patterns that the lexemes of the language tokens may take, using regular expressions
- Regular expressions can be transformed into nondeterministic automata and then into deterministic automata (FSA).
- The latter two notations can be used as input to a "driver", that is code which simulates these automata and uses them as a guide to determining the next token
- This driver and the specification of the automaton form the nucleus of the lexical analyzer
- Lexical analyzers can be generated automatically in a programming language (e.g., C) from a specification of the regular expressions describing the patterns for the tokens, using tools like Lex, Flex, and others.
- The translation rules have the form *Pattern* {*Action*}, where ***Pattern*** is a regular expression and ***Action*** is a code fragment, written in the target programming language for the generated lexical analyzer.
  - ✓ For example such code is needed to access and add entries to the symbol table

Building lexical analyzers with LEX



- A Lex program has the following form

declarations
%%
translation rules
%%
auxiliary functions

Example of Lex specification

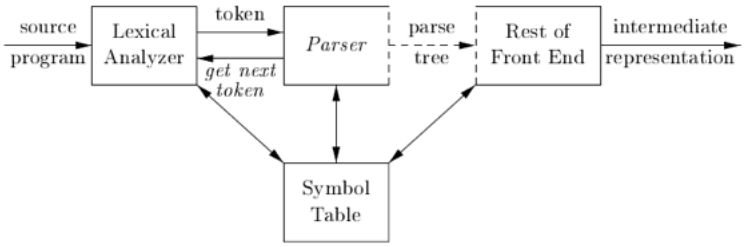
```
%{
/* definition of manifest constants, e.g. ID */
%}
/* regular definitions */
delim [ \t]
ws {delim}+
letter [A-Za-z]
digit [0-9]
id {letter}({letter}|{digit})*
%%
{ws} {/* no action and no return */}
{id} {yylval = (int) installID(); return(ID);}
"+" {return('+');}
"*" {return('*');}
"(" {return('(');}
")" {return(')');}
"\n" {return('\n');}
%%
int installID() {/* function to install the lexeme in the symbol table,
   whose first character is pointed to by yytext, and whose length is
   yyleng, into the symbol table and return a pointer thereto */
}
```

If Lex is used along with Yacc, then it would be normal to define the manifest constants in the Yacc program and use them without definition in the Lex program.
Since lex.yy.c is compiled with the Yacc output, the constants thus will be available to the actions in the Lex program

The attribute value (a numeric code, a pointer to the symbol table, or nothing), is placed in a global variable yylval, which is shared between the lexical analyzer and parser, thereby making it simple to return both the name and an attribute value of a token.

Syntax analysis

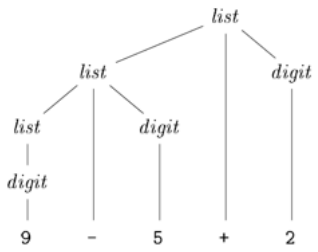
- The syntax of programming language constructs can be specified by context-free grammars or BNF (Backus-Naur Form) notation
- A ***syntax analyzer*** or ***parser*** obtains a string of tokens from the lexical analyzer and verifies that the string of token names can be generated by the (context-free) grammar for the source language
- For well-formed programs, the parser constructs a parse tree and passes it to the rest of the compiler for further processing
  - ✓ The parse tree need not be constructed explicitly, since checking and translation actions can be interspersed with parsing …



Parse Tree

- A parse tree pictorially shows how the start symbol of a grammar derives a string in the language
- Example

$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$



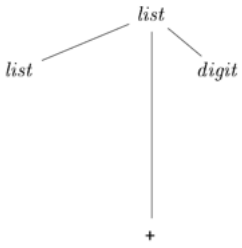
Parse Tree

- Nodes in a parse tree are labeled with terminal or non terminal symbol
  - ✓ Terminal symbols are in the leaves
  - ✓ Non terminal symbols are in the internal nodes
  - ✓ The root node corresponds to the non terminal start symbol
- Each internal node corresponds exactly to one production rule
  - ✓ The parent node is the non terminal symbol that is expanded
  - ✓ The child nodes correspond to the terminal and non terminal symbols in the right side of the production rule. The children are ordered as the corresponding symbols in the production string
- The parse structure is a tree since the production rules have the structure $N \rightarrow \alpha$ that characterizes the context free grammars
  - ✓ The parsed string can be read by a pre-ordered traversal of the tree that outputs only the terminal symbols
- A parse tree can be interpreted as a representation of a sequence of derivations

Parse Tree derivation

- Example: sequence of derivation for (id + id) * id
  - ✓ list ⇒ list + digit

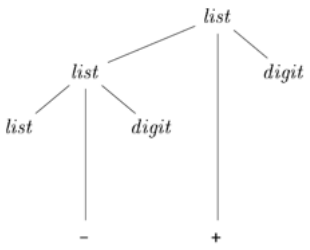
$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$



Parse Tree derivation

- Example: sequence of derivation for (id + id) * id
  - ✓ list ⇒ list + digit ⇒ list – digit + digit

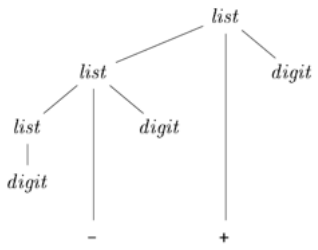
$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$



Parse Tree derivation

- Example: sequence of derivation for (id + id) * id
  - ✓ list ⇒ list + digit ⇒ list – digit + digit ⇒ digit – digit + digit

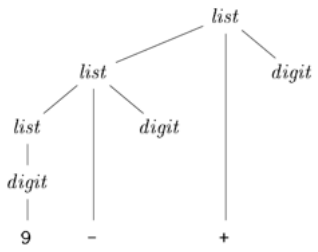
$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$



Parse Tree derivation

- Example: sequence of derivation for (id + id) * id
  - ✓ list ⇒ list + digit ⇒ list – digit + digit ⇒ digit – digit + digit ⇒ 9 – digit +digit

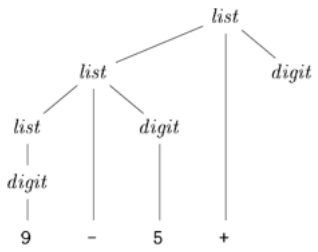
$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$



Parse Tree derivation

- Example: sequence of derivation for (id + id) * id
  - ✓ list ⇒ list + digit ⇒ list – digit + digit ⇒ digit – digit + digit ⇒ 9 – digit +digit ⇒ 9 – 5 + digit

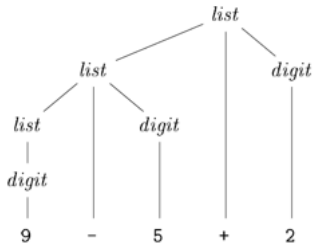
$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$



Parse Tree derivation

- Example: sequence of derivation for (id + id) * id
  - ✓ list ⇒ list + digit ⇒ list – digit + digit ⇒ digit – digit + digit ⇒ 9 – digit +digit ⇒ 9 – 5 + digit ⇒ 9 – 5 + 2
  - ✓ The parse tree shows which production rules were exploited to obtain the derivation but not the order in which they are activated (the same parse tree can be obtained by applying the production rules in a different order)

$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

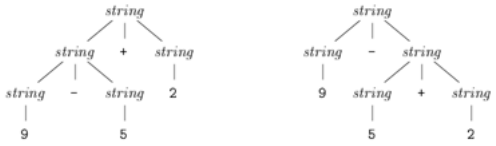


Parse Tree Derivation

- In ***leftmost*** derivations, the leftmost nonterminal in each sentential is always chosen. If $\alpha \Rightarrow \beta$ is a step in which the leftmost nonterminal in α is replaced, we write $\alpha \underset{lm}{\Rightarrow} \beta$
  - ✓ The previous derivation example is leftmost
  - ✓ list ⇒ list + digit ⇒ list – digit + digit ⇒ digit – digit + digit ⇒ 9 – digit + digit ⇒ 9 – 5 + digit ⇒ 9 – 5 + 2
- In ***rightmost*** derivations, the rightmost nonterminal is always chosen; we write $\alpha \underset{rm}{\Rightarrow} \beta$ in this case
  - ✓ list ⇒ list + digit ⇒ list + 2 ⇒ list – digit + 2 ⇒ list – 5 + 2 ⇒ digit – 5 + 2 ⇒ 9 – 5 + 2

Ambiguous grammars

- A grammar is ambiguous if it is possible to build more than one parse tree to describe the derivation process for the same string in the language
- Example
  - ✓ Merge the non terminals list and digit of the previous grammar into one non terminal string
  - ✓ string ⟶ string + string | string - string | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
  - ✓ The expression 9-5+2 has more than one parse tree with this grammar



  - ✓ The two trees correspond to the two ways of parenthesizing the expression: (9-5)+2 and 9-(5+2)
  - ✓ Only the first expression correspond to 9-5+2, the other one produces a different value
  - ✓ The previous grammar does not permit this interpretation (it is not ambigous), as it takes into account the associativity of the operators
- For most parsers, it is desirable that the grammar be made unambiguous

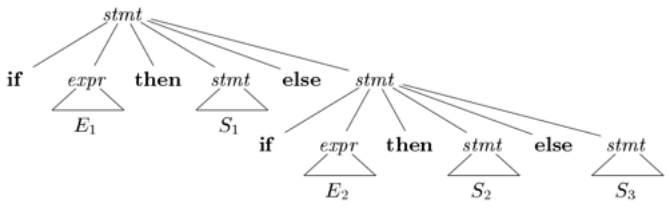
Eliminating ambiguity

- Sometimes an ambiguous grammar can be rewritten to eliminate the ambiguity
- Example

$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \\
 & | & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\
 & | & \textbf{other}
\end{array}
$$

  - ✓ **other** stands for any other statement

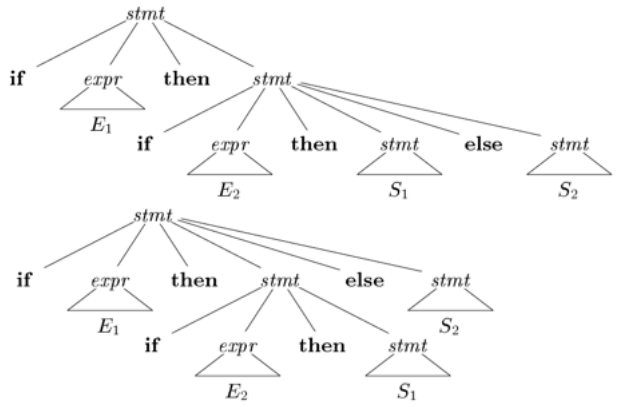
if E_1 **then** S_1 **else if** E_2 **then** S_2 **else** S_3



Eliminating ambiguity

- The grammar is ambigous
- Example
  - ✓ The statement **if** E_1 **then if** E_2 **then** S_1 **else** S_2 has two parse trees:

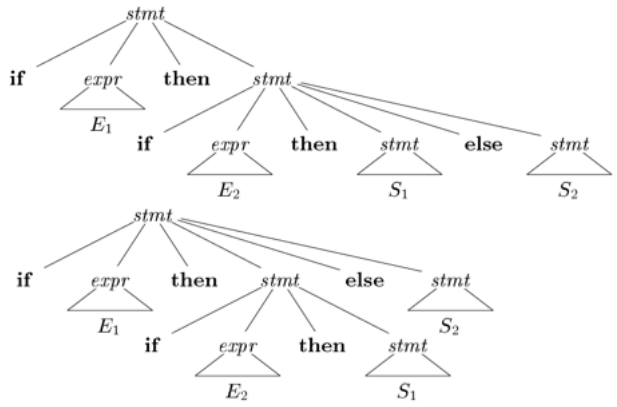
$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \\
 & | & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\
 & | & \textbf{other}
\end{array}
$$



Eliminating ambiguity

- The grammar is ambigous
- Example
  - ✓ The statement **if** E_1 **then if** E_2 **then** S_1 **else** S_2 has two parse trees:

$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if}\ expr\ \textbf{then}\ stmt \\
 & | & \textbf{if}\ expr\ \textbf{then}\ stmt\ \textbf{else}\ stmt \\
 & | & \textbf{other}
\end{array}
$$



In all programming languages with conditional statements of this form, the first parse tree is preferred

"Match each else with the closest unmatched then"

Eliminating ambiguity

- We can rewrite the dangling-else grammar as follows:

$$
\begin{array}{rcl}
\mathit{stmt} & \rightarrow & \mathit{matched_stmt} \\
 & | & \mathit{open_stmt} \\
\mathit{matched_stmt} & \rightarrow & \textbf{if } \mathit{expr} \textbf{ then } \mathit{matched_stmt} \textbf{ else } \mathit{matched_stmt} \\
 & | & \textbf{other} \\
\mathit{open_stmt} & \rightarrow & \textbf{if } \mathit{expr} \textbf{ then } \mathit{stmt} \\
 & | & \textbf{if } \mathit{expr} \textbf{ then } \mathit{matched_stmt} \textbf{ else } \mathit{open_stmt}
\end{array}
$$

- The idea is that a statement appearing between a then and an else must be "matched", that is, the interior statement must not end with an unmatched or open then.
- A matched statement is either an if-then-else statement containing no open statements or it is any other kind of unconditional statement.
- This grammar generates the same strings as the dangling-else grammar, but it allows only one parsing for the previous string, namely, the one that associates each else with the closest previous unmatched then

Context-free grammars vs regular expressions

- Every regular language is a context-free language, according to Chomsky hierarchy
- Example
  - ✓ the expression (**a**|**b**)***abb** and the grammar

$$
\begin{aligned}
A_0 &\rightarrow aA_0 \mid bA_0 \mid aA_1 \\
A_1 &\rightarrow bA_2 \\
A_2 &\rightarrow bA_3 \\
A_3 &\rightarrow \epsilon
\end{aligned}
$$

describe the same language, the set of strings of a's and b's ending in abb

- On the other hand, the language L = $\{a^n\, b^n \mid n \geq 1\}$ with an equal number of *a*'s and *b*'s is a prototypical example of a language that can be described by a context-free grammar, but not by a regular expression
  - ✓ For example, a FSA cannot check whether parentheses are balanced
  - ✓ Colloquially, we say that "finite automata cannot count"

Non context-free language constructs

- Grammars are capable of describing most, but not all, of the syntax of programming languages
  - ✓ A context-free grammar can count two items but not three …
- Example
  - ✓ The language L1 = {*wcw* | *w* is in (**a**|**b**)*} consists of all words composed of a repeated string of *a*'s and *b*'s separated by *c*, such as *aabcaab*.
  - ✓ This language is not context-free, which means that parsers cannot check whether a variable has been declared before being used or distinguish among the strings of different identifiers of arbitrary lenght.
    - » This is the reason why all identifiers are represented by the same token **id**.
- Another example
  - ✓ The language L2 = {$a^n\ b^m\ c^n\ d^m$ | $n \geq 1$ and $m \geq 1$} is not context-free
  - ✓ a^n and b^m could represent the formal-parameter lists of two functions declared to have *n* and *m* arguments, respectively, while c^n and d^m represent the actual-parameter lists in calls to these two functions
- Checking that variables are declared before being used or that actual parameters correspond to formal parameters is done during semantic analysis

Types of parsers

- There are three general types of parsers for grammars: universal, top-down, and bottom-up
  - ✓ Universal parsing methods can parse any grammar. These general methods are, however, too inefficient to use in production compilers
  - ✓ Top-down methods build parse trees from the top (root) to the bottom (leaves), while bottom-up methods start from the leaves and work their way up to the root.
    - » In either case, the input to the parser is scanned from left to right, one symbol at a time.
- The most efficient top-down and bottom-up methods work only for sub-classes of grammars, but several of these classes, particularly, LL and LR grammars, are expressive enough to describe most of the syntactic constructs in modern programming languages
  - ✓ Parsers implemented by hand often use LL grammars
  - ✓ Parsers for the larger class of LR grammars are usually constructed using automated tools

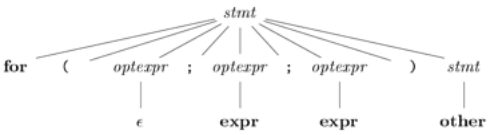
Top-down parsers

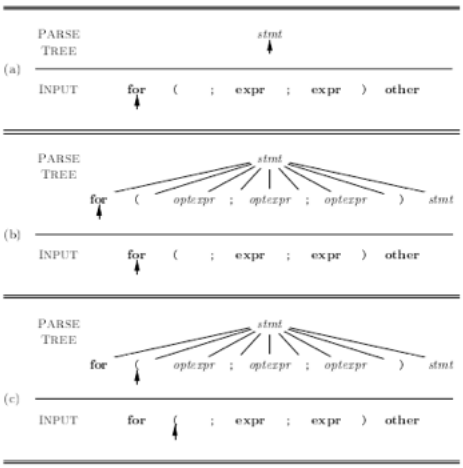
- The top-down construction of a parse tree is done by starting with the root, labeled with the starting and repeatedly performing the following two steps
  - ✓ At node *N*, labeled with nonterminal *A*, select one of the productions for *A* and construct children at *N* for the symbols in the production body
  - ✓ Find the next node at which a subtree is to be constructed, typically the leftmost unexpanded nonterminal of the tree
  - ✓ Top-down parsers build the parse tree according to a leftmost derivation
- For some grammars, the above steps can be implemented during a single left-to-right scan of the input string
  - ✓ The current terminal being scanned in the input is frequently referred to as the ***lookahead symbol***
  - ✓ Initially, the lookahead symbol is the first, i.e., leftmost, terminal of the input string

Example

stmt → **expr ;**
| **if (expr)** *stmt*
| **for (** *optexpr* **;** *optexpr* **;** *optexpr* **)** *stmt*
| **other**

optexpr → ϵ
| **expr**





- Initially, the terminal **for** is the *lookahead symbol*, and the known part of the parse tree consists of the root, labeled with the starting nonterminal *stmt*
- For a match to occur, the nonterminal *stmt* must derive a string that starts with the lookahead symbol **for**
- When the node being considered in the parse tree is a terminal, and the terminal matches the lookahead symbol, then we advance in both the parse tree and the input

Predictive Parsing

- In general, the selection of a production for a nonterminal may involve trial-and-error; that is, we may have to try a production and backtrack to try another production if the first is found to be unsuitable.
  - ✓ A production is unsuitable if, after using the production, we cannot complete the tree to match the input string
  - ✓ Backtracking is not needed for a particular type of top-down parsing, called predictive parsing
- ***Predictive parsing*** is a simple form of ***recursive-descent parsing***, a top-down method of syntax analysis in which a set of recursive procedures (one for each non terminal of the grammar) is used to process the input
- Predictive parsing relies on information about the first symbols that can be generated by a production body
  - ✓ In predictive parsing the lookahead symbol unambiguously determines the flow of control through the procedure body for each nonterminal.
  - ✓ The sequence of procedure calls during the analysis of an input string implicitly defines a parse tree for the input
- Not all grammars are suitable for predictive parsing

Left recursion

- It is possible for a recursive-descent parser to loop forever. A problem arises with "left-recursive" productions like

$$\text{expr} \longrightarrow \text{expr} + \text{term}$$

  - ✓ where the leftmost symbol of the body is the same as the nonterminal at the head of the production.
  - ✓ Suppose the procedure for expr decides to apply this production. The body begins with expr so the procedure for expr is called recursively.
  - ✓ Since the lookahead symbol changes only when a terminal in the body is matched, no change to the input took place between recursive calls of expr. As a result, the second call to expr does exactly what the first call did, which means a third call to expr, and so on, forever.
- In general a grammar is left recursive if it has a nonterminal A such that there is a derivation in one or more steps $A \overset{+}{\Rightarrow} A\alpha$ for some string α.
- Top-down parsing methods cannot handle left-recursive grammars, so a transformation is needed to eliminate left recursion
  - ✓ A left-recursive production can be eliminated by rewriting the offending production.
  - ✓ There are rules and algorithms to eliminate left recursions, including left recursions involving derivations of two or more steps

Eliminating Left recursion: simple case

- Consider a nonterminal A with two productions

$$A \longrightarrow A\alpha \mid \beta$$

where α and β are sequences of terminals and nonterminals that do not start with A

  - ✓ Example: expr ⟶ expr + term | term
  - ✓ A = expr
  - ✓ α = + term
  - ✓ β = term

- The productions above can be replaced by the non-left-recursive productions:

$$A \longrightarrow \beta A'$$
$$A' \longrightarrow \alpha A' \mid \varepsilon$$

$$\begin{array}{lcl} E & \rightarrow & E + T \mid T \\ T & \rightarrow & T * F \mid F \\ F & \rightarrow & (E) \mid \mathbf{id} \end{array} \quad \Longrightarrow \quad \begin{array}{lcl} E & \rightarrow & T\ E' \\ E' & \rightarrow & +\ T\ E' \mid \epsilon \\ T & \rightarrow & F\ T' \\ T' & \rightarrow & *\ F\ T' \mid \epsilon \\ F & \rightarrow & (E) \mid \mathbf{id} \end{array}$$

Left factoring

- Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive, or top-down, parsing
- When the choice between two alternative A-productions is not clear, we may be able to rewrite the productions to defer the decision until enough of the input has been seen that we can make the right choice
- Example

$$\begin{array}{lcl} stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\ & | & \textbf{if } expr \textbf{ then } stmt \end{array}$$

  - ✓ on seeing the input if, we cannot immediately tell which production to choose to expand stmt
- If $A \longrightarrow \alpha\,\beta_1 \mid \alpha\,\beta_2$ are two A-productions, and the input begins with a nonempty string derived from α, we do not know whether to expand A to $\alpha\,\beta_1$ or $\alpha\,\beta_2$.
- However, we may defer the decision by expanding A to $\alpha A'$. Then, after seeing the input derived from α, we expand A' to β_1 or to β_2.
  - ✓ The productions can be rewritten as

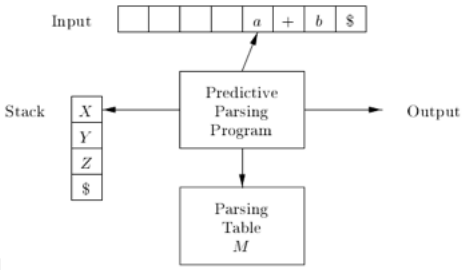
$$\begin{array}{l} A \rightarrow \alpha A' \\ A' \rightarrow \beta_1 \mid \beta_2 \end{array}$$

Predictive Parsing and LL(1) grammars

- Predictive parsers can be constructed for a class of grammars called LL(1).
  - ✓ The first "L" stands for scanning the input from left to right,
  - ✓ The second "L" stands for producing a leftmost derivation
  - ✓ "1" stands for using one input symbol of lookahead at each step to make parsing action decisions
- The class of LL(1) grammars is rich enough to cover most programming constructs, although care is needed in writing a suitable grammar for the source language
  - ✓ For example, no left-recursive or ambiguous grammar can be LL(1)

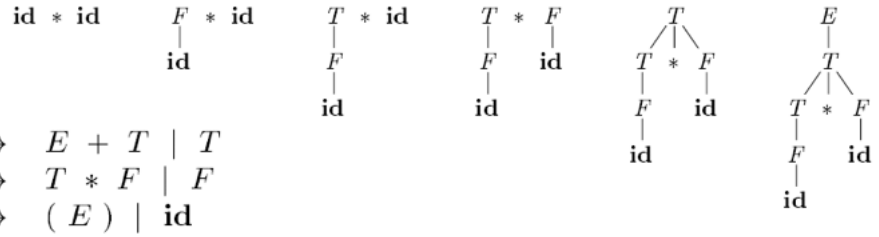
Nonrecursive Predictive Parsing

- A nonrecursive predictive parser can be built by maintaining a stack explicitly, rather than implicitly via recursive calls
- The input buffer contains the string to be parsed, followed by the endmarker $.
- The symbol $ is also used to mark the bottom of the stack, which initially contains the start symbol of the grammar on top of $.
- The parser is controlled by a program that considers X, the symbol on top of the stack, and a, the current input symbol
  - ✓ If X is a nonterminal, the parser chooses an X-production by consulting entry $M[X, a]$ of the parsing table M and push the symbols of the body of the production to the stack (from right to left)
  - ✓ Otherwise, it checks for a match between the terminal X and current input symbol a and in this case it pop X from the stack and advances on the input string



Bottom-up Parsing

- A bottom-up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (top).
  - ✓ It is convenient to describe parsing as the process of building parse trees, although a front end may in fact carry out a translation directly without building an explicit tree

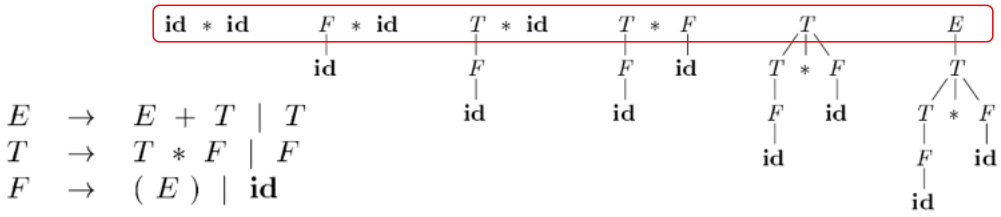


$$
\begin{aligned}
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \mathbf{id}
\end{aligned}
$$

- We can think of bottom-up parsing as the process of "reducing" a string w to the start symbol of the grammar.

Reductions

- By definition, a reduction is the reverse of a step in a derivation
- At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.
- The key decisions during bottom-up parsing are about when to reduce and about what production to apply, as the parse proceeds
- Example:



$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid \mathbf{id}$$

- The goal of bottom-up parsing is to construct a rightmost derivation in reverse during a left-to-right scan of the input
  - ✓ $E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$

Handle Pruning

- A rightmost derivation in reverse can be obtained by ***handle pruning***
  - ✓ A ***handle*** is a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation.
- Let $S = \gamma_0 \underset{rm}{\Rightarrow} \gamma_1 \underset{rm}{\Rightarrow} \gamma_2 \underset{rm}{\Rightarrow} \ldots \underset{rm}{\Rightarrow} \gamma_{n-1} \underset{rm}{\Rightarrow} \gamma_n = w$ be the (yet unknown) rightmost derivation for w
  - ✓ To reconstruct this derivation in reverse order, we locate the handle β_n in γ_n and replace β_n by the head of the relevant production $A_n \longrightarrow \beta_n$ to obtain the previous right-sentential form γ_{n-1} (and repeat this process until we get a right-sentential form consisting of the start symbol S)

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \mathbf{id}
\end{array}
$$

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \rightarrow \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \rightarrow F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \rightarrow \mathbf{id}$
$T * F$	$T * F$	$T \rightarrow T * F$
T	T	$E \rightarrow T$

  - ✓ Note that although T is the body of the production E ⟶ T, the symbol T is not a handle in the sentential form T * $\mathbf{id}_2$. If T were indeed replaced by E, we would get the string E * $\mathbf{id}_2$, which cannot be derived from the start symbol E

Shift-Reduce Parsing

- Shift-reduce parsing is a form of bottom-up parsing in which a stack holds grammar symbols and an input buffer holds the rest of the string to be parsed
  - ✓ We use $ to mark the bottom of the stack and also the right end of the input.
- Initially, the stack is empty, and the string w is on the input

STACK	INPUT
$	w $

- During a left-to-right scan of the input string, the parser shifts zero or more input symbols onto the stack, until it is ready to reduce a string β of grammar symbols on top of the stack.
- It then reduces β to the head of the appropriate production
  - ✓ the symbols composing the string β are popped from the stack and the head of the production is pushed on the stack
- The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty

STACK	INPUT
$ S	$

Shift-Reduce Parsing Example

- Parsing the input string $\mathbf{id}_1$ * $\mathbf{id}_2$
  - ✓ $E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \mathbf{id}
\end{array}
$$

STACK	INPUT	ACTION
$	$\mathbf{id}_1 * \mathbf{id}_2$ $	shift
$ $\mathbf{id}_1$	$* \mathbf{id}_2$ $	reduce by $F \rightarrow \mathbf{id}$
$ F	$* \mathbf{id}_2$ $	reduce by $T \rightarrow F$
$ T	$* \mathbf{id}_2$ $	shift
$ $T *$	$\mathbf{id}_2$ $	shift
$ $T * \mathbf{id}_2$	$	reduce by $F \rightarrow \mathbf{id}$
$ $T * F$	$	reduce by $T \rightarrow T * F$
$ T	$	reduce by $E \rightarrow T$
$ E	$	accept

- While the primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make: (1) shift, (2) reduce, (3) accept, and (4) error

Shift-Reduce Parsing Conflicts

- There are context-free grammars for which shift-reduce parsing cannot be used
  - ✓ Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and also the next k input symbols, cannot decide whether to shift or to reduce (a ***shift/reduce conflict***), or cannot decide which of several reductions to make (a ***reduce/reduce conflict***).
- Example of shift/reduce conflict

$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \\
 & | & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\
 & | & \textbf{other}
\end{array}
$$

  - ✓ If we have a shift-reduce parser in configuration

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

we cannot tell whether if expr then stmt is the handle.

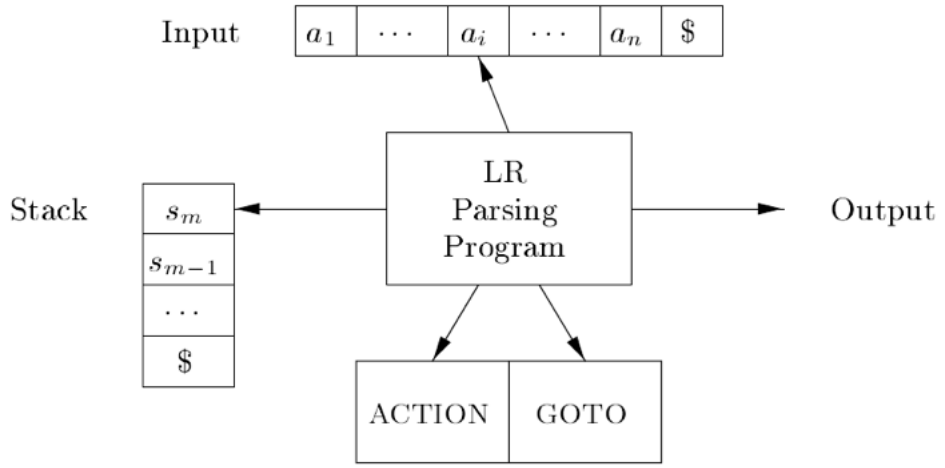
LR parsing and LR grammars

- The most prevalent type of bottom-up parser today is based on a concept called LR(k) parsing
  - ✓ "L" stands for left-to-right scanning of the input
  - ✓ "R" stands for constructing a rightmost derivation in reverse,
  - ✓ k stands for the number of input symbols of lookahead that are used in making parsing decisions (when k is omitted k is assumed to be 1)
- LR-parsing is the most general nonbacktracking shift-reduce parsing method known
  - ✓ LR parsers can be constructed to recognize virtually all programming language constructs for which context-free grammars can be written
- Intuitively, for a grammar to be LR it is sufficient that a left-to-right shift-reduce parser be able to recognize handles of right-sentential forms when they appear on top of the stack
- LR parsers are table-driven, much like the nonrecursive LL parsers.
  - ✓ The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive or LL methods

LR parser construction

- Constructing an LR parser by hand for a typical programming-language grammar is very hard.
- A specialized tool, an LR parser generator (like Yacc), is needed, which takes a context-free grammar and automatically produces a parser for that grammar
  - ✓ If the grammar contains ambiguities or other constructs that are difficult to parse in a left-to-right scan of the input, then the parser generator locates these constructs and provides detailed diagnostic messages
- The LR parser generator builds a ***pushdown automaton*** from the LR grammar
- Based on the current state and on the current input symbol the automaton decides which actions (shift/reduce) to take and what is the next state
  - ✓ A state remembers how much of a production we have seen at a given point in the parsing process
  - ✓ The transitions of the automaton and the actions to be made by the parsers are stored in a parsing table
  - ✓ The LR parser uses a stack to push a state (instead of the symbol) when a shift is made and pop one or more states when a reduction is made

LR parser

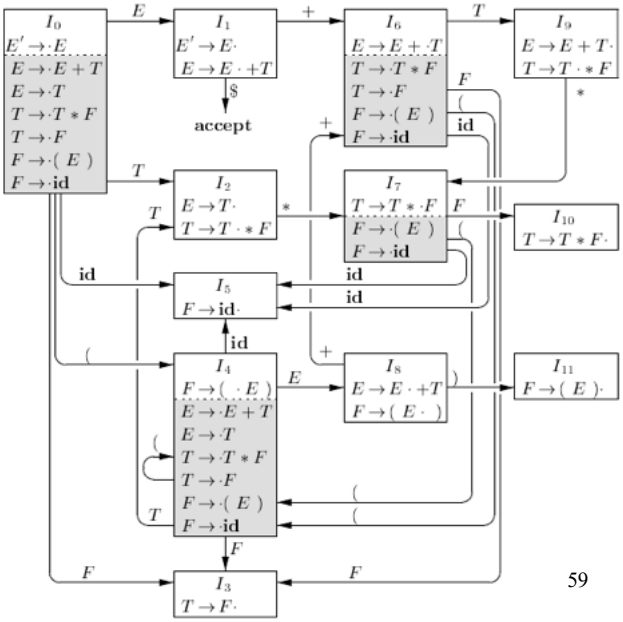


Parsing Table

LR automaton example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$

- A grammar *G* with start symbol *S* is augumented with a production *S'* ⟶ *S* to indicate to the parser when it should stop parsing and announce ***acceptance*** of the input
- States represent sets of ***items***, where an item is a production with a dot at some position of the body (indicating how much of the production we have seen in that state)
- A state includes the ***closure*** of a set of items.

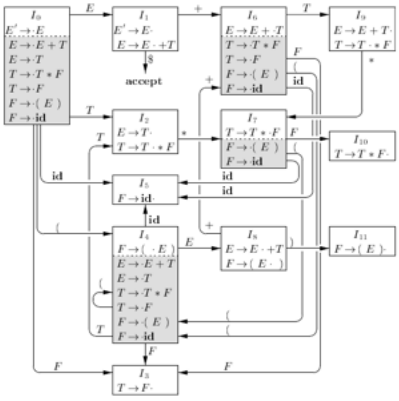


LR parsing table (ACTION and GOTO)

- The ACTION function takes the state s on top of the stack and the current symbol a to decide the action (shift/reduce/accept/error)
  - ✓ ACTION[s, a] = Shift t. The action represents a transition from state s to state t on the automaton. The state t is pushed onto the stack (it represents pushing the input symbol a onto the stack) and the the parser advances on the input string
  - ✓ ACTION[s, a] = Reduce $A \longrightarrow \beta$. The action represents a reduction of β to head A. A number of states equal to the lenght of β are popped from the stack (it represents popping the symbols of the body of the production from the stack).
- The function GOTO is used after a reduce action is made to decide next transition of the automaton.
  - ✓ Let t be the state on top of the stack (after the reduction $A \longrightarrow \beta$ popped $|\beta|$ states from the stack).
  - ✓ The function GOTO takes the state t on top of the stack and the head A of the production used in the reduction
  - ✓ If GOTO[t, A] = s then the state s is pushed onto the stack (it represents pushing the head of the production onto the stack)

LR parsing table example

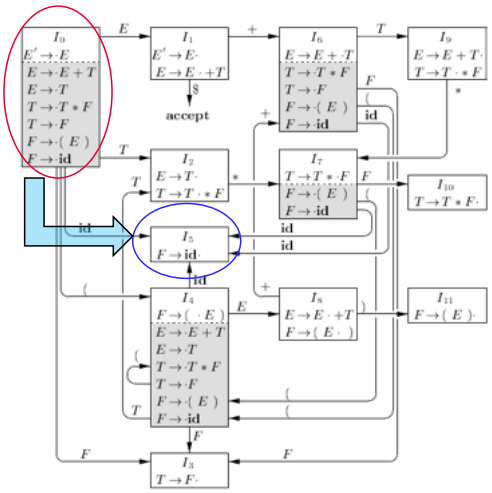
(1) $E \to E + T$
(2) $E \to T$
(3) $T \to T * F$
(4) $T \to F$
(5) $F \to (E)$
(6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

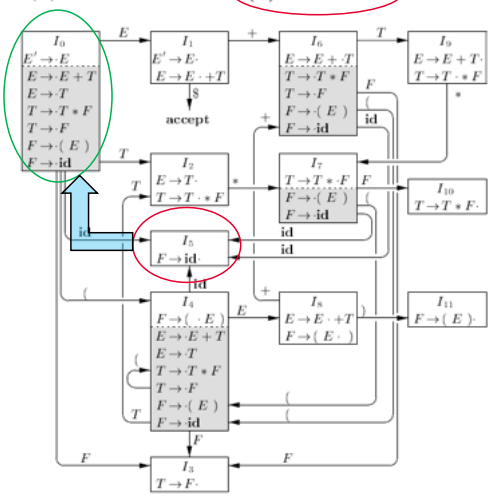
Stack = $ I_0
String = • **id** + **id** **$**

Stack = $ I_0 I_5
String = **id** • + **id** **$**

Action(0, id) = s5

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

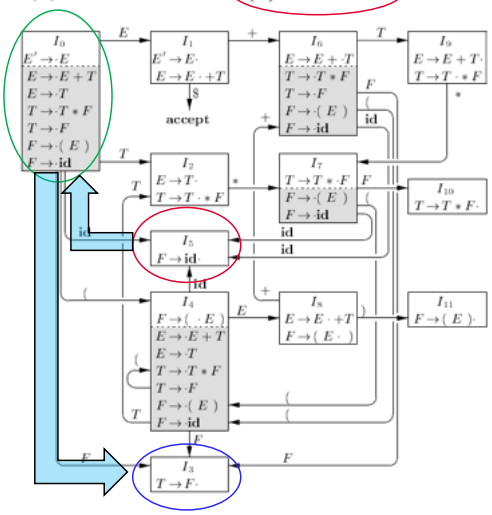
Stack = $ I_0 I_5
String = **id** • + **id $**

Stack = $ I_0
String = **id** • + **id $**

Action(5, +) = r6 (F ⟶ id)

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $ I_0 I_5
String = **id** • + **id $**

Action(5, +) = r6 (F ⟶ id)

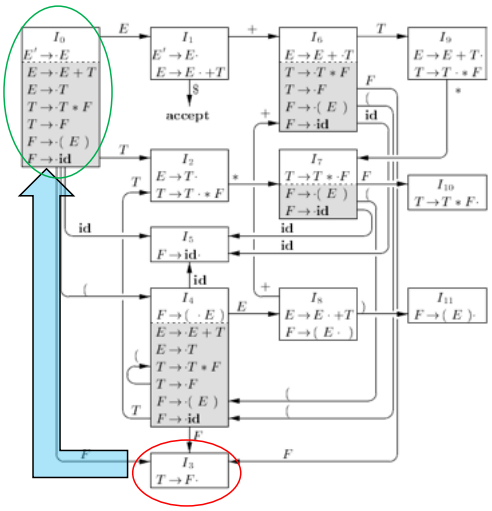
Stack = $ I_0
String = **id** • + **id $**

goto(0, F) = 3

Stack = $ I_0 I_3
String = **id** • + **id $**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

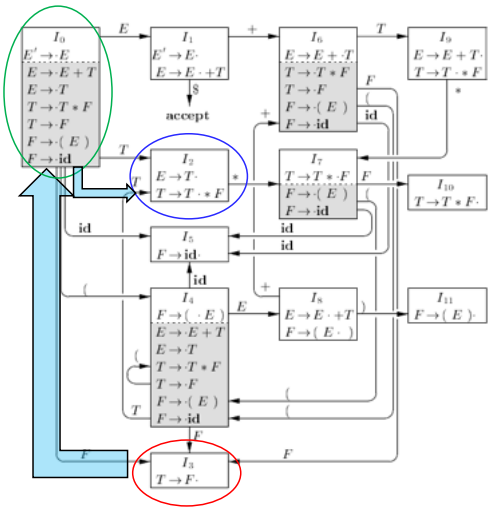
Stack = $ I_0 I_3
String = **id** • + **id $**

Stack = $ I_0
String = **id** • + **id $**

Action(3, +) = r4 (T ⟶ F)

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $ I_0 I_3
String = **id** • + **id $**

Action(3, +) = r4 (T ⟶ F)

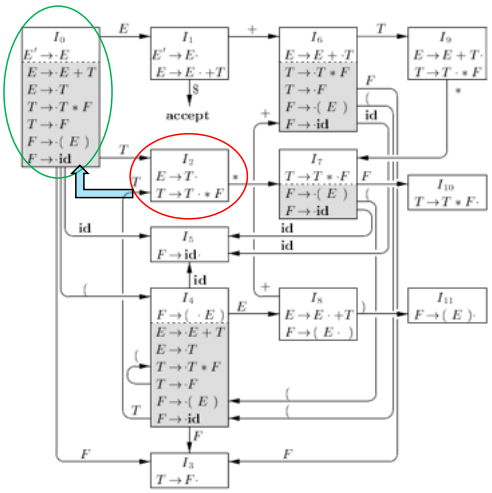
Stack = $ I_0 I_2
String = **id** • + **id $**

Stack = $ I_0
String = **id** • + **id $**

goto(0, T) = 2

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

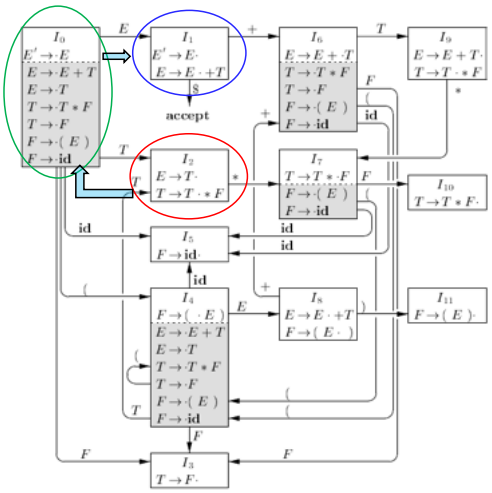
Stack = $ I_0 I_2
String = **id** • + **id $**

Stack = I_0
String = **id** • + **id $**

Action(2, +) = r2 (E ⟶ T)

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $ I_0 I_2
String = **id** • + **id $**

Action(2, +) = r2 (E ⟶ T)

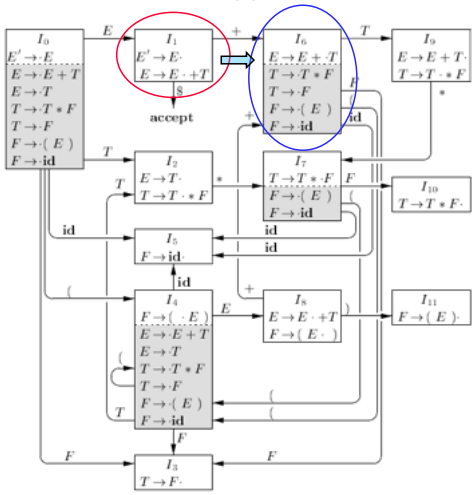
Stack = I_0 I_1
String = **id** • + **id $**

Stack = I_0
String = **id** • + **id $**

goto(0, E) = 1

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



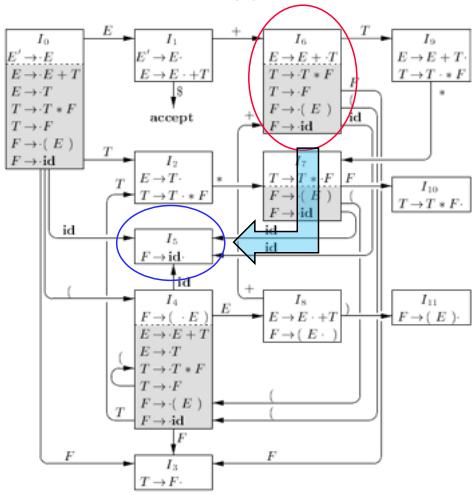
STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $I_0 I_1$
String = **id** • + **id $**
Action(1, +) = s6

Stack = $I_0 I_1 I_6$
String = **id** + • **id $**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

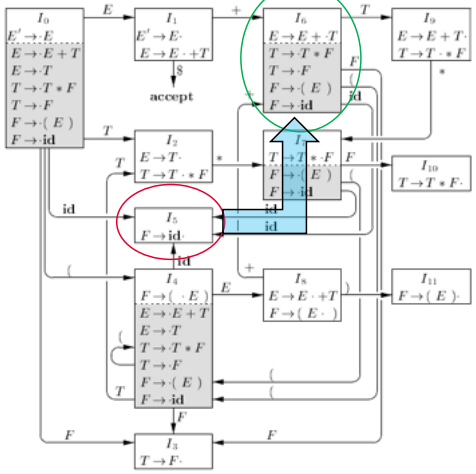
Stack = $I_0\, I_1\, I_6$
String = **id** + • **id $**

Stack = $I_0\, I_1\, I_6\, I_5$
String = **id** + **id** • $

Action(6, id) = s5

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

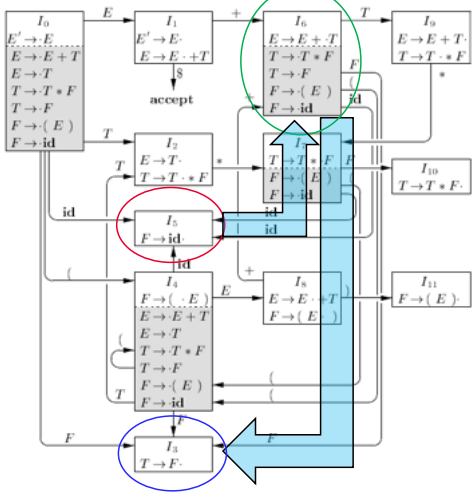
Stack = $I_0\, I_1\, I_6\, I_5$
String = **id** + **id** • $

Action(5, $) = r6 (F ⟶ id)

Stack = $I_0\, I_1\, I_6$
String = **id** + **id** • **$**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

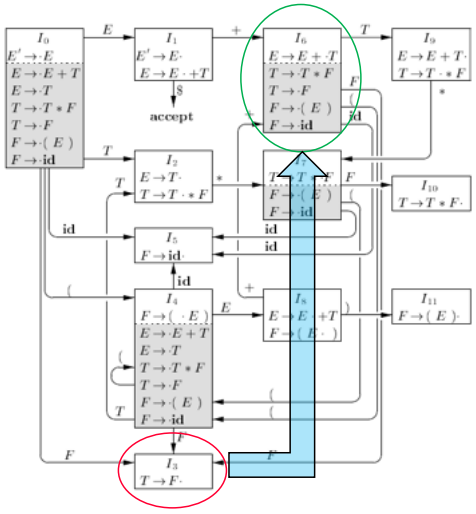
Stack = $I_0\, I_1\, I_6\, I_5$
String = **id** + **id** • $
Action(5, $) = r6 (F ⟶ id)

Stack = $I_0\, I_1\, I_6$
String = **id** + **id** • **$**
goto(6, F) = 3

Stack = $I_0\, I_1\, I_6\, I_3$
String = **id** + **id** • **$**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



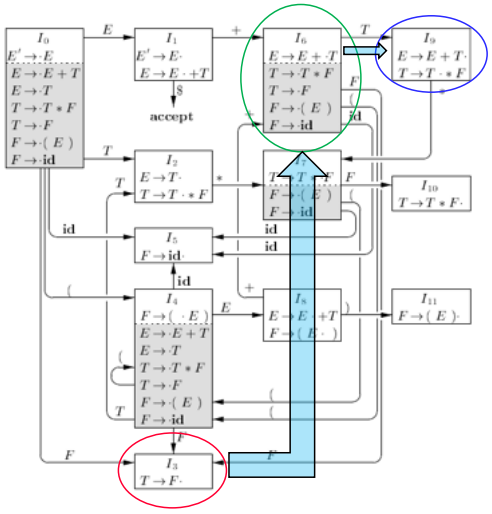
STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $I_0\, I_1\, I_6\, I_3$
String = **id** + **id** • **$**
Action(3, $) = r4 (T ⟶ F)

Stack = $I_0\, I_1\, I_6$
String = **id** + **id** • **$**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

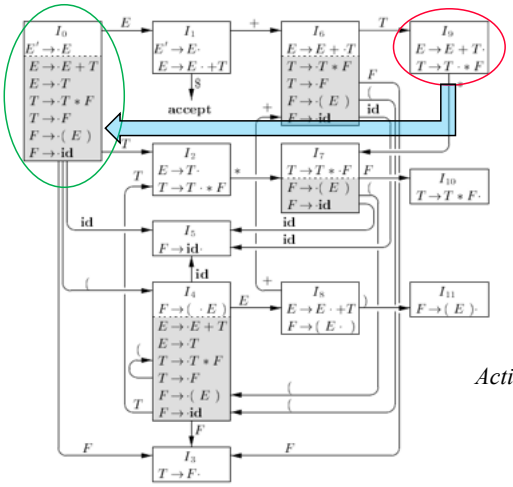
Stack = $I_0\, I_1\, I_6\, I_3$
String = **id** + **id** • **$**
Action(3, $) = r4 (T ⟶ F)

Stack = $I_0\, I_1\, I_6$
String = **id** + **id** • **$**
goto(6, T) = 9

Stack = $I_0\, I_1\, I_6\, I_9$
String = **id** + **id** • **$**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

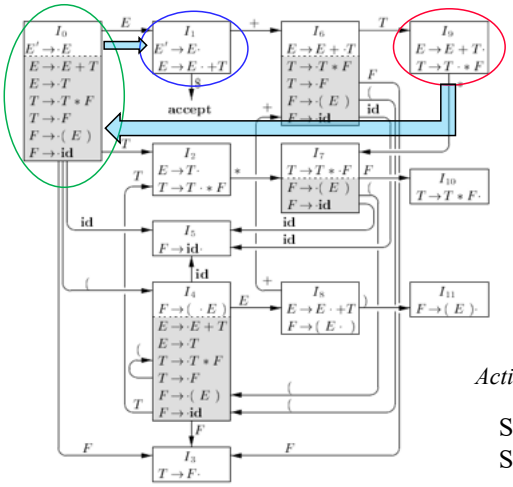
Stack = $I_0 I_1 I_6 I_9$
String = **id** + **id** • **$**

Action(9, $) = r1 (E ⟶ E + T)

Stack = I_0
String = **id** + **id** • **$**

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$



STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $I_0\, I_1\, I_6\, I_9$
String = **id** + **id** • **$**

Action(9, $) = r1 (E ⟶ E + T)

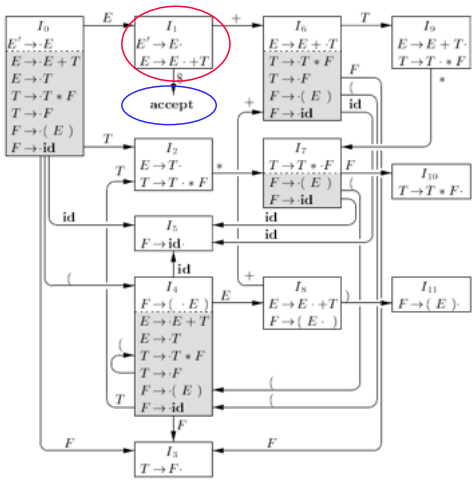
Stack = $I_0\, I_1$
String = **id** + **id** • **$**

Stack = I_0
String = **id** + **id** • **$**

goto(0, E) = 1

LR parsing algorithm example

(1) $E \to E + T$ (4) $T \to F$
(2) $E \to T$ (5) $F \to (E)$
(3) $T \to T * F$ (6) $F \to \mathbf{id}$

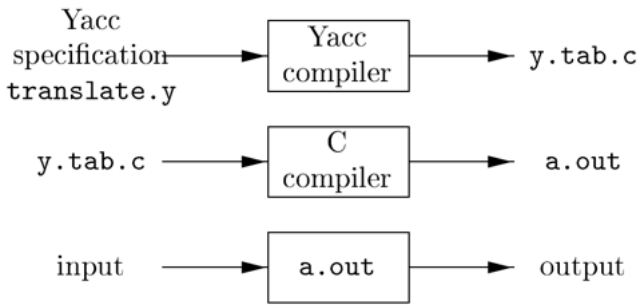


STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack = $I_0\ I_1$
String = **id** + **id** • **$**

Action(1, $) = Accept

Parser generator YACC



- YACC is a LALR (*lookahead*-LR) parser generator
  - ✓ LALR method is often used in practice, because it generates smaller parsing tables, yet most common syntactic constructs of programming languages can be expressed conveniently by an LALR grammar

Example of YACC specification

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \textbf{digit}
\end{array}
$$

```
%{
#include <ctype.h>
%}
%token DIGIT
%%
line : expr '\n' { printf("%d\n", $1); }
;
expr : expr '+' term { $$ = $1 + $3; }
| term
;
term : term '*' factor { $$ = $1 * $3; }
| factor
;
factor : '(' expr ')' { $$ = $2; }
| DIGIT
;
%%
```

By default $$ = $1 in case the body of a production contains only one symbol

```
yylex() {
   int c;
   c = getchar();
   if (isdigit(c)) {
      yylval = c-'0';
      return DIGIT;
   }
   return c;
}
```

We can also use Lex to pass the tokens to the Yacc parser.

$$ refers to the attribute value associated with the nonterminal of the head, while $i refers to the value associated with the ith grammar symbol (terminal or nonterminal) of the body.

Revised YACC specification

```
%{
#include <ctype.h>
%}
%token DIGIT
%%
line : expr '\n' { printf("%d\n", $1); }
;
expr : expr '+' term { $$ = $1 + $3; }
| term
;
term : term '*' factor { $$ = $1 * $3; }
| factor
;
factor : '(' expr ')' { $$ = $2; }
| DIGIT
;
%%
#include "lex.yy.c "
```

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \textbf{digit}
\end{array}
$$

Lex specification

```
%{
/* no need to define DIGIT as it is defined as token in YACC
   specification*/
%}
/* regular definitions */
delim [ \t]
ws {delim}+
digit [0-9]
%%
{ws} {/* no action and no return */}
{digit} {yylval = c-'0'; return(DIGIT);}
"+" {return('+');}
"*" {return('*');}
"(" {return('(');}
")" {return(')');}
"\n" {return('\n');}
%%
```

Syntax Directed Translation

- Syntax-directed translation is done by attaching rules or program fragments to productions in a grammar
  - ✓ We associate information with a language construct by attaching attributes to the (terminals and nonterminals) grammar symbols representing the construct
- A syntax-directed definition specifies the values of attributes by associating semantic rules with the grammar productions.
  - ✓ By convention, semantic actions are enclosed within curly braces
  - ✓ The position of a semantic action in a production body determines the order in which the action is executed
- The most general approach to syntax-directed translation is to construct a syntax tree, and then to compute the values of attributes at the nodes of the tree by visiting the nodes of the tree
  - ✓ In many cases, translation can be done during parsing, without building an explicit tree

Syntax Directed Definitions

- A *syntax-directed definition* (SDD) is a context-free grammar together with attributes and rules.
  - ✓ Attributes are associated with grammar symbols and rules are associated with productions.
  - ✓ If X is a symbol and a is one of its attributes, then we write $X.a$ to denote the value of a at a particular parse-tree node labeled X.
  - ✓ If we implement the nodes of the parse tree by records or objects, then the attributes of X can be implemented by data fields that represent the nodes for X
- Attributes may be of any kind: numbers, types, table references, strings, …
  - ✓ The strings may even be long sequences of code, say code in the intermediate language used by a compiler

Inherited and Synthesized Attributes

- A *synthesized attribute* for a nonterminal A at a parse-tree node N is defined by a semantic rule associated with the production at N.
  - ✓ The production must have A as its head.
  - ✓ A synthesized attribute at node N is defined only in terms of attribute values at the children of N and at N itself.
- An *inherited attribute* for a nonterminal B at a parse-tree node N is defined by a semantic rule associated with the production at the parent of N.
  - ✓ The production must have B as a symbol in its body.
  - ✓ An inherited attribute at node N is defined only in terms of attribute values at N's parent, N itself, and N's siblings
- Terminals can have synthesized attributes, but not inherited attributes.
  - ✓ Attributes for terminals have lexical values that are supplied by the lexical analyzer; there are no semantic rules in the SDD itself for computing the value of an attribute for a terminal

S-attributed SDD

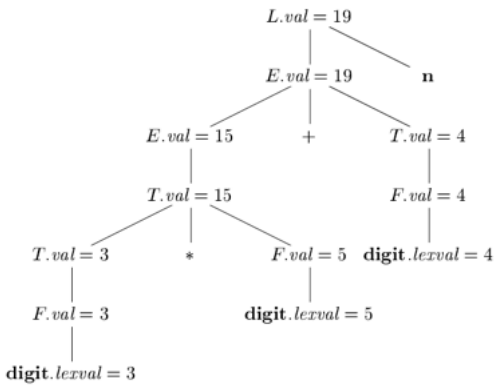
	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E$ **n**	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow$ **digit**	$F.val =$ **digit**.lexval

lexval is an integer value returned by the lexical analyzer

- An SDD that involves only synthesized attributes is called *S-attributed*
- In an S-attributed SDD, each rule computes an attribute for the nonterminal at the head of a production from attributes taken from the body of the production
- S-attributed definitions are suitable for bottom up parsing

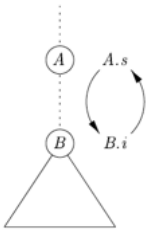
Annotated Parse Tree

- An *annotated parse tree* is a parse tree showing the value(s) of its attribute(s)



For SDD's with both inherited and synthesized attributes, there is no guarantee that there is even one order in which to evaluate attributes at nodes.

PRODUCTION	SEMANTIC RULES
$A \rightarrow B$	$A.s = B.i;$ $B.i = A.s + 1$



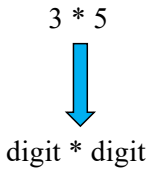
With synthesized attributes, we can evaluate attributes in any bottom-up order, such as that of a postorder traversal of the parse tree

L-attributed SDD

- In *L-attributed definitions* dependencies between attribute values can go from left to right and not from right to left
  - ✓ They are suitable for top-down parsing
- Each attribute must be either synthesized or inherited, but with the rules limited as follows.
- Suppose that there is a production $A \longrightarrow X_1 X_2 \cdots X_n$, and there is an inherited attribute $X_i.a$ computed by a rule associated with this production. Then the rule may use only:
  - ✓ Inherited attributes associated with the head A.
  - ✓ Either inherited or synthesized attributes associated with the occurrences of symbols X_1 $X_2 \cdots X_{i-1}$ located to the left of X_i.
  - ✓ Inherited or synthesized attributes associated with this occurrence of X_i itself, but only in such a way that there are no cycles in a dependency graph formed by the attributes of this X_i.

Example

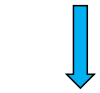
	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$



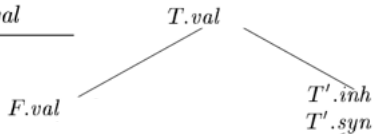
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



digit * digit



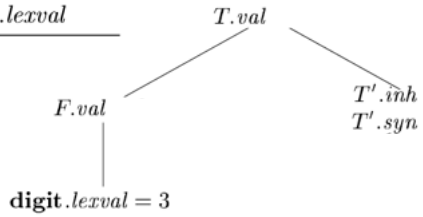
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



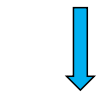
digit * digit



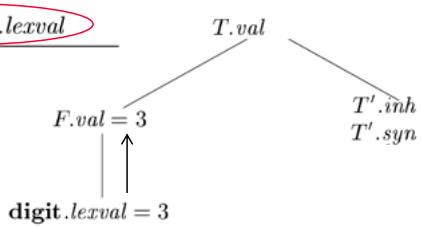
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



digit * digit



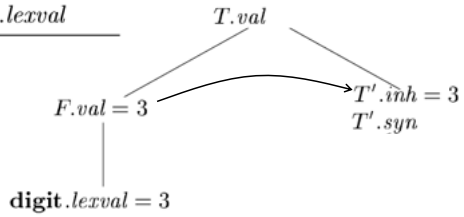
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



digit * digit



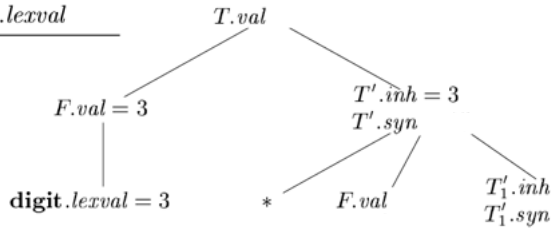
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



digit * digit



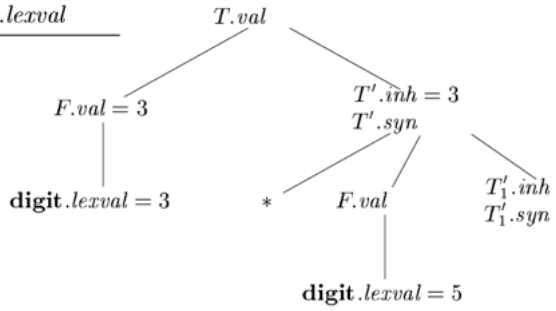
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$

3 * 5

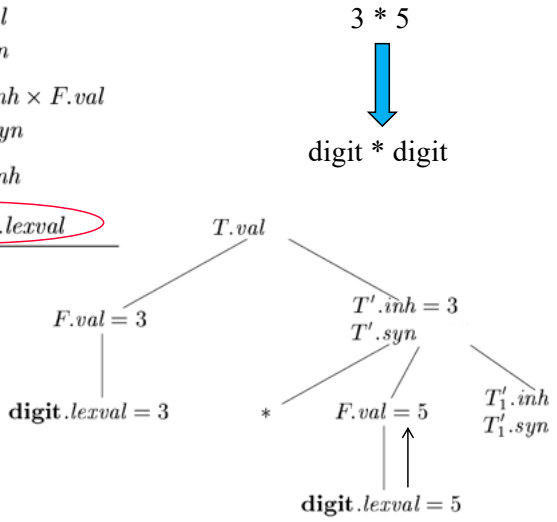


digit * digit



Example

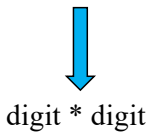
	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow$ **digit**	$F.val =$ **digit**$.lexval$



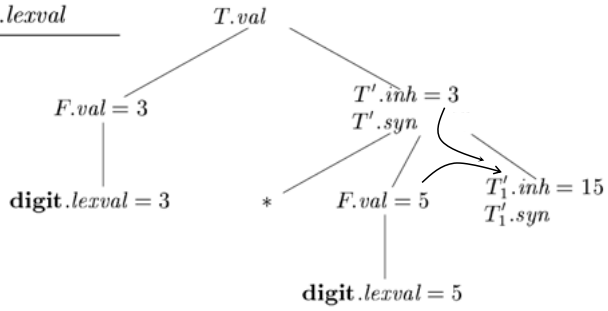
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



digit * digit

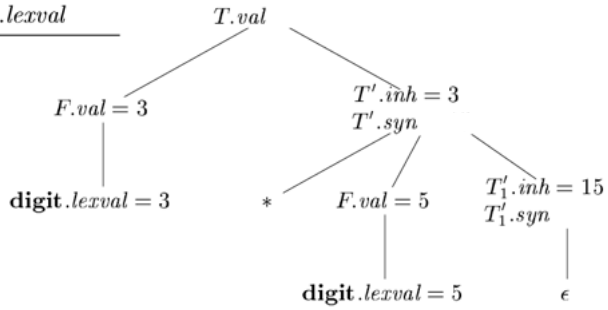


Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

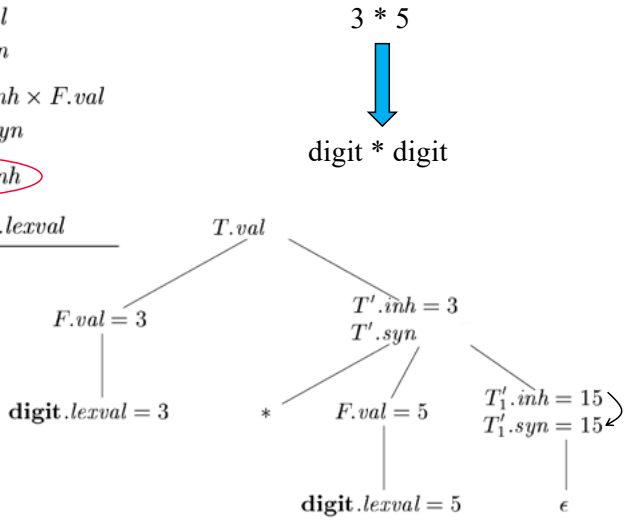
3 * 5

digit * digit



Example

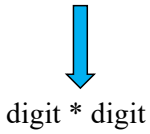
	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$



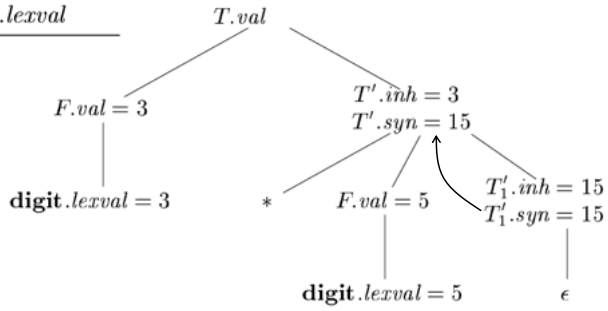
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\, T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow * F\, T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



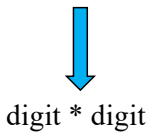
digit * digit



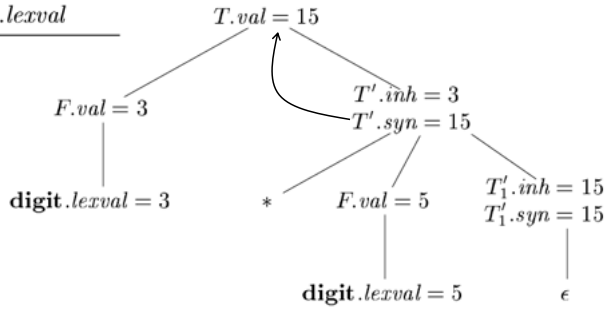
Example

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

3 * 5



digit * digit



Applications of SDD

- One of the main applications of SDD is the construction of syntax trees
  - ✓ Some compilers use syntax trees as an intermediate representation
  - ✓ To complete the translation to intermediate code, the compiler may then walk the syntax tree, using another set of rules that are in effect an SDD on the syntax tree rather than the parse tree.
- Each node in a syntax tree represents a construct, the children of the node represent the meaningful components of the construct
  - ✓ A syntax-tree node representing an expression E1 + E2 has label + and two children representing the subexpressions E1 and E2.

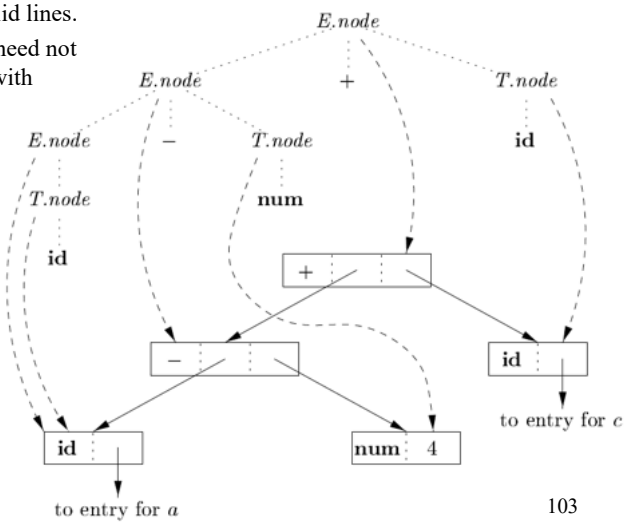
Construction of Syntax Trees

- Nodes of a syntax tree are implemented by objects (or records) with a suitable number of fields, including an *op* field that is the label of the node and additional fields
  - ✓ If the node is a leaf, an additional field holds the lexical value for the leaf. A constructor function *Leaf* (*op*, *val*) creates a leaf object. Alternatively, if nodes are viewed as records, then *Leaf* returns a pointer to a new record for a leaf.
  - ✓ If the node is an interior node, there are as many additional fields as the node has children in the syntax tree. A constructor function *Node* takes two or more arguments: *Node*(*op*, c_1, c_2, …, c_k) creates an object with first field *op* and *k* additional fields (pointers to nodes) for the *k* children c_1, c_2, …, c_k.

	PRODUCTION	SEMANTIC RULES
1)	$E \rightarrow E_1 + T$	$E.node = \textbf{new}\ Node('+', E_1.node, T.node)$
2)	$E \rightarrow E_1 - T$	$E.node = \textbf{new}\ Node('-', E_1.node, T.node)$
3)	$E \rightarrow T$	$E.node = T.node$
4)	$T \rightarrow (\ E\ )$	$T.node = E.node$
5)	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
6)	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.val)$

Example

- Construction of a syntax tree for the input $a - 4 + c$
  - ✓ The node of the syntax tree are shown as records, with the *op* field first.
  - ✓ Syntax-tree edges are shown as solid lines.
  - ✓ The underlying parse tree, which need not actually be constructed, is shown with dotted edges
  - ✓ The dashed lines represent the values of *E.node* and *T.node*; each line points to the appropriate syntax-tree node

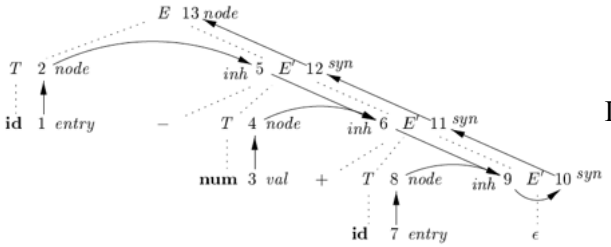


Sequence of steps

1) $p_1 = \textbf{new}\ \mathit{Leaf}(\textbf{id}, \mathit{entry\text{-}a});$
2) $p_2 = \textbf{new}\ \mathit{Leaf}(\textbf{num}, 4);$
3) $p_3 = \textbf{new}\ \mathit{Node}('-', p_1, p_2);$
4) $p_4 = \textbf{new}\ \mathit{Leaf}(\textbf{id}, \mathit{entry\text{-}c});$
5) $p_5 = \textbf{new}\ \mathit{Node}('+', p_3, p_4);$

Construction of Syntax Trees during top-down parsing

	PRODUCTION	SEMANTIC RULES
1)	$E \rightarrow T\ E'$	$E.node = E'.syn$ $E'.inh = T.node$
2)	$E' \rightarrow +\ T\ E'_1$	$E'_1.inh = \textbf{new}\ Node('+', E'.inh, T.node)$ $E'.syn = E'_1.syn$
3)	$E' \rightarrow -\ T\ E'_1$	$E'_1.inh = \textbf{new}\ Node('-', E'.inh, T.node)$ $E'.syn = E'_1.syn$
4)	$E' \rightarrow \epsilon$	$E'.syn = E'.inh$
5)	$T \rightarrow (\ E\ )$	$T.node = E.node$
6)	$T \rightarrow \textbf{id}$	$T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)$
7)	$T \rightarrow \textbf{num}$	$T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.val)$



- Nonterminal *E*' has an inherited attribute *inh* and a synthesized attribute *syn*.
- Attribute *E*'.*inh* represents the partial syntax tree constructed so far
- Specifically, it represents the root of the tree for the prefix of the input string that is to the left of the subtree for *E*'.

Dependency graph for $a - 4 + c$

Intermediate code generation and semantic analysis

- In the process of translating a program in a given source language into code for a given target machine, a compiler may construct a sequence of intermediate representations, starting from the syntax tree



- During this phase a number of analyses can be performed, such as type checking, control flow analysis, and data flow analysis
  - ✓ Note that some representations (e.g., control flow graphs with information about defined and used variables at a given node), can either be derived by visiting the syntax tree or be directly constructed during syntax directed translation (e.g., control flow edges can be added to the nodes of the syntax tree)
- Integrated Development Environment, typically provides some APIs to visit the syntax tree and extract other intermediate representations to be used for different types of analysis.